★ QuizOps

# Passo a passo: QuizOps na AWS

FIAP · Cloud Computing · Multicloud

---

Como recriar a aplicação e a arquitetura do zero em outra conta AWS. O Terraform cria a infraestrutura e o GitHub Actions faz o build e o deploy do front e do back. Nada é compilado na sua máquina.

## Pré-requisitos

- Conta AWS com permissão de administrador
- Conta no GitHub
- AWS CLI v2
- Terraform 1.5+
- GitHub CLI autenticado (`gh auth login`)
- Git

## 1. Fazer o fork e clonar

No GitHub, faça o fork de `luizbrito7/gs02-multicloud.fiap` e clone o seu fork. Ref.: fork de um repositório.

```
git clone https://github.com/<seu-usuario>/gs02-multicloud.fiap.git
cd gs02-multicloud.fiap
```

## 2. Configurar o profile da AWS

```
aws configure --profile meu-profile
aws sts get-caller-identity --profile meu-profile
```

O segundo comando deve mostrar o ID da sua conta. Ref.: perfis do AWS CLI.

## 3. Ajustar o Terraform

Em `terraform/00-provider.tf`, troque o profile e o nome do projeto:

```
locals {
  project = "quiz-rm12345"   # nome único: vira parte do nome dos buckets
  ...
}

provider "aws" {
  profile = "meu-profile"
  ...
}
```

Nome de bucket S3 é único no mundo inteiro, por isso o `project` precisa ser diferente do original. Ref.: regras de nome de bucket.

## 4. Criar a infraestrutura

```
cd terraform
terraform init
terraform plan -out tfplan
terraform apply tfplan
```

Leva alguns minutos (o CloudFront é o mais lento). No fim, o Terraform mostra os outputs: `api_url`, `user_pool_id`, `user_pool_client_id`, `front_url` e outros. A Lambda nasce sem as dependências; o pipeline do passo 6 publica o código completo. Ref.: terraform apply.

## 5. Preparar o pipeline

1. No fork, abra a aba **Actions** e habilite os workflows (em fork eles vêm desligados). Ref.: habilitar workflows.
2. Em **Settings** → **Secrets and variables** → **Actions**, crie `AWS_ACCESS_KEY_ID` e `AWS_SECRET_ACCESS_KEY`. Ref.: secrets no GitHub Actions.
3. Em `.github/workflows/cicd-front.yml`, troque `front-quiz-lab` por `front-<seu project>-lab` (duas vezes).
4. Em `.github/workflows/cicd-back.yml`, troque `lambda-quiz-lab` por `lambda-<seu project>-lab`.

5. Ainda dentro de `terraform/`, aponte o front para a sua infraestrutura criando as Variables do repositório. O `cicd-front` passa essas variáveis para o build; arquivos `.env` não vão para o Git.

```
gh variable set VITE_API_URL --body "$(terraform output -raw api_url)"
gh variable set VITE_USER_POOL_ID --body "$(terraform output -raw user_pool_id)"
gh variable set VITE_USER_POOL_CLIENT_ID --body "$(terraform output -raw
user_pool_client_id)"
gh variable list
```

Se a infraestrutura mudar, rode os mesmos comandos de novo. Ref.: gh variable set, variables no GitHub Actions, variáveis de ambiente no Vite.

## 6. Fazer o deploy pelo pipeline

```
cd ..
git add .
git commit -m "chore: aponta o projeto para a minha conta AWS"
git push
```

Em **Actions**, rode os dois pipelines manualmente na `main` (**Run workflow**). Ref.: rodar um workflow manualmente.

- **Front (`cicd-front`):** build com as Variables do passo 5, envio ao S3 e invalidação do CloudFront.
- **Back (`cicd-back`):** empacota o código com as dependências e atualiza a Lambda.

Espere os dois ficarem verdes.

## 7. Testar

1. Abra a URL do output `front_url` (`https://xxxx.cloudfront.net`).
2. Crie uma conta e confirme com o código que chega no e-mail. Ref.: cadastro no Cognito.
3. Responda o quiz, envie uma foto no mural e confira o ranking.

## 8. Liberar um novo quiz (opcional)

Crie `app/backend/quizzes/02.json` no formato do `01.json`, com `"released": true`, e faça push na `main`. O `cicd-back` roda sozinho e atualiza a Lambda.

Se um `terraform apply` futuro atualizar a `aws_lambda_function` (aparece no plan), rode o `cicd-back` de novo para publicar as dependências.

## 9. Destruir tudo

```
cd terraform
terraform destroy
```

Os buckets têm `force_destroy`, então são apagados mesmo com arquivos. Ref.: terraform destroy.